

Integracja ładowarek AMPERE POINT z Home Assistant przez xtend_tuya

Raport z naprawy konfiguracji, wyjaśnienie działania systemu
oraz poradnik dalszych kroków

wersja 1 — 5 lipca 2026 — dokument wewnętrzny projektu AMPERE_POINT

Spis treści

1 Cel i zakres dokumentu	3
2 Stan wyjściowy i objawy problemu	3
3 Diagnoza — skąd wiadomo, co było nie tak	4
4 Wykonane kroki — chronologicznie, z wyjaśnieniami	4
4.1 Etap A — dodanie integracji Xtend Tuya w Home Assistant	4
4.2 Etap B — projekt na platformie deweloperskiej Tuya	5
4.3 Etap C — odblokowanie wszystkich punktów danych (DP Instruction) .	6
4.3.1 Odkrycie: „AmperePoint EV Charger” to produkt zarejestrowany jako źródło światła	7
5 Prawdopodobna przyczyna wcześniejszych niepowodzeń	7
6 Jak to naprawę działa — architektura, sieci, protokoły	8
6.1 Home Assistant od środka	8
6.2 Droga danych: ładowarka → chmura Tuya → Home Assistant	8
6.3 Platforma deweloperska Tuya — co czym jest	9
6.4 Rola xtend_tuya i dodatku firmowego	10
7 Dokończenie konfiguracji — krok po kroku	10
8 Poradnik: następne kroki — weryfikacja, energia, automatyzacje	11
8.1 Porządki po konfiguracji	11
8.2 Panel Energia	11
8.3 Dodatek firmowy TuyaExtend AmperePoint	11
8.4 Automatyzacje — jak myśleć i trzy przykłady	11
8.5 Pulpit (dashboard)	12
8.6 Kopie zapasowe i aktualizacje	12
8.7 Kierunek na przyszłość: sterowanie lokalne	13
9 Przewidywane problemy i proponowane rozwiązania	13

10 Słowniczek pojęć i skrótów**15**

1. Cel i zakres dokumentu

Dokument podsumowuje sesję z 5 lipca 2026, w której doprowadzono do działania rozszerzoną integrację ładowarek AMPERE POINT z Home Assistant. Opisuje kolejno: stan wyjściowy i objawy problemu, wykonane kroki wraz z uzasadnieniem każdego z nich, najbardziej prawdopodobną przyczynę wcześniejszych niepowodzeń, zasadę działania całego łańcucha (ładowarka — chmura Tuya — Home Assistant) razem z użytymi sieciami i protokołami, instrukcję dokończenia konfiguracji, poradnik automatyzacji oraz listę przewidywanych problemów z proponowanymi rozwiązaniami. Na końcu znajduje się słowniczek pojęć i skrótów.

Terminy techniczne są wyjaśniane przy pierwszym użyciu, a najważniejsze z nich zebrano dodatkowo w słowniczku (rozdział 10).

2. Stan wyjściowy i objawy problemu

Punkt wyjścia w dniu sesji wyglądał następująco:

- **Home Assistant** (dalej: HA) — system automatyki domowej, czyli oprogramowanie zbierające w jednym miejscu dane z urządzeń i pozwalające nimi sterować — działał na komputerze w **Dockerze**. Docker to narzędzie uruchamiające programy w tzw. kontenerach, czyli odizolowanych „pudełkach” zawierających program razem ze wszystkim, czego potrzebuje do działania. Wcześniejsza próba uruchomienia HA w maszynie wirtualnej VirtualBox (system HAOS) została porzucona z powodu problemów sieciowych maszyny wirtualnej; wariant z Dockerem okazał się stabilny. Interfejs HA był dostępny w przeglądarce pod adresem `localhost:8123`, a pliki konfiguracyjne w katalogu `C:\HomeAssistant\config`.
- **HACS** (Home Assistant Community Store — społecznościowy „sklep” z dodatkami, których nie ma w oficjalnym HA) był zainstalowany i sprawny.
- **Oficjalna integracja Tuya** (integracja = moduł HA łączący go z konkretnym systemem lub usługą) była skonfigurowana i widziała **10 urządzeń** na koncie `dawid.cekala@gmail.com`: AmperePoint EV Charger, EV Charger VE, EV Charging Station, GPEV100, Q11 OTA, Q11 OTA 2 (ładowarka_q11), Q22 OTA, Q22 OTA 2, Q37 OTA oraz urządzenie_zepsute (Q22 OTA).
- Każde urządzenie miało jednak tylko **jedną encję** — przełącznik (Switch). Encja to pojedyncza „zmienna” widoczna w HA: jeden czujnik, jeden przełącznik, jedna wartość. Brakowało encji prądu, mocy, energii, napięcia, statusu ładowania itd.
- Rozszerzenie **xtend_tuya** w wersji 4.4.8 (dodatek społecznościowy autora `azerty9971`, który dokłada brakujące encje do oficjalnej integracji Tuya) było **pobrane w HACS**, ale — jak się okazało — nigdy nie zostało skutecznie **dodane jako integracja**. To rozróżnienie jest kluczowe i wraca w rozdziale 5.
- W katalogu `custom_components` znajdował się też firmowy dodatek **TuyaExtend AmperePoint** — pomocnik, który nie pobiera żadnych danych z Tuya, lecz jedynie mapuje i przelicza encje już istniejące w HA (np. koszt ładowania z taryfą w PLN). Bez encji źródłowych nie miał czego mapować.

Objaw końcowy: mimo pobrania xtend_tuya liczba encji nie rosła, a wejście w pozycję „Xtend Tuya” w HACS pokazywało jedynie opis (README) wyglądający jak instrukcja instalacji.

3. Diagnoza — skąd wiadomo, co było nie tak

Diagnozę oparto na dwóch źródłach: transkrypcie wcześniejszej rozmowy „Home assistant (historia)” oraz sześciu zrzutach ekranu z 5 lipca (folder home_assistant w projekcie). Kluczowe obserwacje:

1. Zrzut listy integracji (Ustawienia → Urządzenia i usługi) pokazywał: Tuya, HACS, Backup, prognozę pogody i kilka drobnych — **ale żadnego wpisu „Xtend Tuya”**. Pobranie w HACS kopiuje tylko pliki na dysk; encje powstają dopiero, gdy integracja zostanie dodana i utworzy się jej **wpis konfiguracyjny** (config entry). Brak wpisu = brak encji, niezależnie od tego, ile razy restartowano system.
2. Zrzut HACS potwierdzał, że pakiet Xtend Tuya v4.4.8 jest pobrany (sekcja „Pobrane”).
3. Zrzut strony urządzenia (urządzenie_zepsute, Q22 OTA) potwierdzał objaw: jedna encja Switch, brak czujników.
4. Transkrypt wcześniejszej rozmowy pokazywał, że podczas prób dodawania integracji pojawiało się „jakieś dodatkowe okno”, które zostało zamknięte. To okno okazało się ostatnim krokiem kreatora konfiguracji — szczegóły w rozdziale 5.

Wniosek diagnostyczny: pliki były na miejscu, konto Tuya działało, a jedynym brakującym ogniwem było **skuteczne przejście kreatora dodawania integracji do samego końca**.

4. Wykonane kroki — chronologicznie, z wyjaśnieniami

Prace podzielono na trzy etapy zgodne z dokumentacją xtend_tuya: **A** — dodanie integracji bez chmury deweloperskiej, **B** — podłączenie platformy deweloperskiej Tuya (pełniejsze dane), **C** — odblokowanie wszystkich punktów danych urządzeń. Wszystkie kliknięcia w przeglądarce wykonywał asystent (Chrome, sterowanie zdalne); logowania, hasła i kody QR obsługiwał Dawid.

4.1. Etap A — dodanie integracji Xtend Tuya w Home Assistant

1. **Ustawienia → Urządzenia i usługi → Dodaj integrację**, wyszukanie „xtend”. Na liście pojawiły się obie pozycje: **Xtend Tuya** oraz **TuyaExtend AmperePoint (helper)** — dowód, że pliki obu dodatków były poprawnie zainstalowane i ładowane przez HA.
2. Wybrano **Xtend Tuya**. Kreator poprosił o **kod użytkownika** (User Code) — to identyfikator konta Tuya, *nie* hasło. Znajduje się w aplikacji mobilnej: **Smart Life → Ja → Ustawienia (zębátka) → Konto i bezpieczeństwo → Kod użytkownika**. Wielkość liter ma znaczenie.

3. Po zatwierdzeniu kodu kreator wyświetlił **kod QR**, który Dawid zeskanował aplikacją Smart Life (Ja → ikona skanera) i zatwierdził logowanie na telefonie. To ten sam mechanizm, którego używa oficjalna integracja Tuya — HA „podszywa się” za dodatkową aplikację na koncie.
4. **Moment krytyczny:** po zatwierdzeniu QR kreator pokazał formularz „**Add Tuya OpenAPI credentials**” (dodaj dane dostępowe do OpenAPI — interfejsu programistycznego chmury Tuya). Przy pierwszym podejściu w tej sesji formularz został zamknięty krzyżykiem — i cały kreator przepadł bez komunikatu: wpis integracji nie powstał. **To odtworzyło dokładnie wcześniejszy problem** (rozdział 5).
5. Procedurę powtórzono. Tym razem w formularzu zaznaczono pole „**Don't ask for OpenAPI credentials**” (nie pytaj o dane OpenAPI), bo kluczy z platformy deweloperskiej jeszcze nie było. Uwaga na pułapkę: mimo zaznaczenia tego pola formularz **nadal wymagał** wybrania kraju i punktu końcowego (endpoint — adres serwera API); bez tego zgłaszał błąd walidacji „value must be one of [...]”. Wybrano **Country = Poland** oraz **endpoint = Europe** (czyli <https://openapi.tuya.eu.com> — centrum danych Europa Środkowa) i zatwierdzono.
6. Kreator przeszedł do okna „Name and assign” (nazwy urządzeń i przypisanie do obszarów) — zostawiono wartości domyślne i kliknięto **Zakończ**.

Efekt etapu A: na liście integracji pojawił się wpis **Xtend Tuya — 10 urządzeń**, a łączna liczba encji wzrosła z 10 do około **154**:

Urządzenie	encje przed	encje po etapie A
Q22 OTA	1	24
Q22 OTA 2	1	24
urządzenie_zepsute (Q22 OTA)	1	24
GPEV100	1	16
Q11 OTA	1	15
ładowarka_q11 (Q11 OTA)	1	15
Q37 OTA	1	15
EV Charger VE	1	15
EV Charging Station	1	5
AmperePoint EV Charger	1	1

Urządzenie „AmperePoint EV Charger” nie zyskało nic — wyjaśnienie tego przypadku znajduje się w punkcie 4.3.1.

4.2. Etap B — projekt na platformie deweloperskiej Tuya

Celem etapu B jest danie xtend_tuya drugiego, pełniejszego kanału dostępu do chmury Tuya — przez **OpenAPI**, czyli oficjalny interfejs programistyczny dla deweloperów. Wymaga to darmowego konta na <https://iot.tuya.com> (platforma deweloperska Tuya; po zalogowaniu przekierowuje na platform.tuya.com).

1. Dawid zalogował się na platformę (konto deweloperskie zakłada się samodzielnie; można użyć tego samego adresu e-mail co w aplikacji).

2. Utworzono projekt chmurowy: **Cloud** → **Development** → **Create Cloud Project**. Wypełniono: nazwa **AmperePoint HA**, branża (Industry) **Smart Home**, metoda rozwoju (Development Method) **Smart Home**, centrum danych (Data Center) **Central Europe Data Center**. Wybór centrum danych jest **krytyczny**: musi odpowiadać regionowi konta aplikacji Smart Life (dla Polski — Europa Środkowa). Darmowy plan (Trial Edition usługi IoT Core) pozwala na jedno centrum danych.
3. Kreator „Authorize API Services” zaproponował pięć usług, które zatwierdzono bez zmian: **IoT Core** (rdzeń — dostęp do urządzeń; w pakiecie Free Basic Resource Pack), **Authorization Token Management** (zarządzanie tokenami, czyli tymczasowymi „przepustkami” do API), **Smart Home Basic Service**, **Data Dashboard Service** oraz **Smart Home Scene Linkage** (oznaczona jako przestarzała, nieszkodliwa).
4. Po utworzeniu projektu na karcie **Overview** dostępne są klucze projektu (sekcja *Authorization Key*): **Access ID/Client ID** (identyfikator: dne8yn9jnwta5gxtrsh) oraz **Access Secret/Client Secret** (sekret — ukryty pod ikoną oka; pełni rolę hasła do API i nie należy go nikomu udostępniać ani zapisywać w miejscach współdzielonych). Kod projektu: p1783262455403t77xgh.
5. Powiązано konto aplikacji z projektem: zakładka **Devices** → **Link App Account** → **Add App Account** → **Tuya App Account Authorization**. Na ekranie pojawił się kod QR, który Dawid zeskanował aplikacją Smart Life i zatwierdził. Wybrano tryb **Automatic Link** (automatyczne dołączanie wszystkich urządzeń konta — także przyszłych). Platforma potwierdziła dołączenie **10 urządzeń** konta dawid.cekala@gmail.com.

Efekt etapu B: projekt deweloperski widzi wszystkie ładowarki (np. ładowarka_q11 — Online), a klucze Client ID/Secret są gotowe do wpisania w HA. Samo wpisanie kluczy pozostało do wykonania po stronie Dawida (rozdział 7) — asystent zasadniczo nie wprowadza cudzych kluczy API.

4.3. Etap C — odblokowanie wszystkich punktów danych (DP Instruction)

Chmura Tuya udostępnia urządzenia przez API na dwa sposoby: **Standard Instruction** (instrukcje standardowe — ujednolicone, ale okrojone mapowanie funkcji dla danej kategorii urządzeń) albo **DP Instruction** (surowe **punkty danych**). Punkt danych (DP, data point) to pojedyncza zmienna urządzenia — np. „prąd fazy L1”, „energia sesji”, „stan pracy” — o numerze, nazwie kodowej i typie. Tryb standardowy potrafi ukrywać część DP, dlatego dokumentacja xtend_tuya zaleca przełączenie produktów na tryb DP.

Procedura (dla **każdej** kategorii produktu osobno): zakładka **Devices** → **All Devices**, przycisk „...” przy pasku „View Devices by Product” → najechać kursorem na kafelek produktu → ikona **ołówka** → zaznaczyć **DP Instruction** → **Save Configuration** → potwierdzić **OK**. Ostrzeżenie platformy („devices might not be able to be controlled by the original standard instructions”) dotyczy wyłącznie tego projektu API — nie wpływa na aplikację Smart Life ani na działanie ładowarek.

Stan na moment przerwania sesji (Dawid przejął dokańczanie): przełączone **EV Charger_VE**, **AmperePoint EV Charger** i **Q20_3.5KW_with_OTA**; do

przełączenia pozostały **EV Charging Station, Q11_11KW_with_OTA, Q22 OTA i GPEV100**.

4.3.1. Odkrycie: „AmperePoint EV Charger” to produkt zarejestrowany jako źródło światła

Strona konfiguracji trybu instrukcji ujawniła, że produkt **AmperePoint EV Charger** (PID, czyli identyfikator produktu: gbmngxngploofmhbjc) ma w chmurze Tuya typ „**Light Source**” (źródło światła) i **jeden jedyny punkt danych** `switch_led`. To nie jest błąd HA ani `xtend_tuya` — **producent zarejestrował ten produkt w złej kategorii i z minimalnym zestawem DP**. Dlatego urządzenie ma i będzie miało tylko jedną encję, dopóki producent nie poprawi definicji produktu. Zalecenie: zgłosić to firmie (podając PID) — warto poprosić o przeniesienie do kategorii ładowarek EV z pełnym zestawem punktów danych.

5. Prawdopodobna przyczyna wcześniejszych niepowodzeń

Najbardziej prawdopodobna przyczyna, potwierdzona „na żywo” w tej sesji, jest prozaiczna i **konstrukcyjna, nie użytkowa**: kreator Xtend Tuya po zeskanowaniu kodu QR wyświetla **dodatkowy formularz** „Add Tuya OpenAPI credentials”. Formularz wygląda jak opcjonalna, „doklejona” prośba o dane deweloperskie — ale jego **zamknięcie krzyżykiem przerywa cały kreator**: wpis integracji nie powstaje, a HA nie pokazuje żadnego błędu ani ostrzeżenia. Użytkownik ma poczucie, że „wszystko zrobił” (pobrał, zrestartował, dodał, zeskanował QR) — i obiektywnie tak było, z wyjątkiem jednego okna, którego funkcja jest kompletnie nieoczywista. W tej sesji zamknięcie tego formularza odtworzyło problem w 100%: po zamknięciu wpis integracji nie istniał; po ponownym przejściu kreatora i *obsłudze* formularza (checkbox + kraj + endpoint) — wszystko zadziało od razu.

Czynniki, które dodatkowo utrudniały wcześniejsze próby:

- **Formularz ma błąd walidacji**: mimo zaznaczenia „Don’t ask for OpenAPI credentials” wymaga wybrania kraju i endpointu; komunikat błędu (lista adresów serwerów) nie wyjaśnia, czego dotyczy.
- **Mylący ekran w HACS**: pozycja Xtend Tuya pokazuje README z przyciskiem „OPEN HACS REPOSITORY”, który niczego nie instaluje; właściwy przycisk „Download” znika po pobraniu, co sugerowało awarię.
- **Fałszywy trop z forkami**: adresy `amperepoint/tuya-ap` i `dawidcekała-oss/tuya-ap` zwracały 404, bo repozytorium firmowe jest prywatne, a HACS czyta tylko publiczne — stąd wcześniejsza instalacja ręczna dodatku firmowego do `custom_components`.
- **Restart nie mógł pomóc**: bez wpisu integracji restartowanie kontenera nic nie zmienia, co budowało wrażenie „nic nie działa mimo wszystkiego”.

Krótko: **nie było żadnego błędu w instalacji plików ani w koncie Tuya — brakowało jednego, źle zaprojektowanego kroku kreatora, który po cichu anulował całą konfigurację**.

6. Jak to naprawdę działa — architektura, sieci, protokoły

6.1. Home Assistant od środka

Home Assistant to program napisany w języku Python, zbudowany wokół trzech pojęć:

Encja (entity)

pojedyncza wielkość lub element sterowania: czujnik mocy, przełącznik, pole wyboru trybu. Każda encja ma identyfikator (np. `sensor.q22_ota_moc`), bieżący **stan** (np. 7.4) i **atrybuty** (np. jednostkę kW). Zmiany stanów płyną przez wewnętrzną **magistralę zdarzeń** (event bus) i są zapisywane do lokalnej bazy danych (domyślnie SQLite — plikowa baza w katalogu konfiguracji), dzięki czemu działa zakładka Historia.

Urządzenie (device)

grupa encji pochodzących z jednego fizycznego sprzętu (np. ładowarka Q22 = kilkanaście encji). To tylko „teczka” porządkująca — logika siedzi w encjach.

Integracja (integration)

moduł tłumaczący świat zewnętrzny (chmurę, protokół, urządzenie) na encje. Integracja po skonfigurowaniu tworzy **wpis konfiguracyjny** (config entry) zapisywany w ukrytym katalogu `.storage`; wpis pamięta konto, tokeny itd.

Pliki integracji bez wpisu konfiguracyjnego nie robią nic — to była istota naszego problemu. Kreator, który taki wpis tworzy, nazywa się **config flow**.

Dodatki społecznościowe (jak `xtend_tuya`) trafiają do katalogu `custom_components` — HA ładuje je przy starcie obok integracji wbudowanych. **HACS** to tylko wygodny instalator: pobiera pliki z serwisu GitHub (publiczne repozytoria kodu) i przypomina o restarcie; nie konfiguruje niczego.

Warstwa systemowa: HA działa w kontenerze Dockera. Kontener współdzieli system operacyjny gospodarza, ale ma własny, odizolowany system plików i sieć; katalog `C:\HomeAssistant\config` jest „wpięty” (zamontowany) do kontenera jako `/config`, więc konfiguracja przeżywa restarty i aktualizacje. Port 8123 kontenera jest opublikowany na komputerze — stąd adres `http://localhost:8123` (localhost = „ten komputer”). Na Windows Docker Desktop uruchamia kontenery w lekkiej maszynie wirtualnej z Linuksem (WSL2), co jest niewidoczne w codziennym użyciu. Restart HA = restart kontenera `homeassistant` w Docker Desktop; w tej wersji instalacji HA nie ma własnego przycisku „Uruchom ponownie” w menu System.

6.2. Droga danych: ładowarka → chmura Tuya → Home Assistant

Ładowarki AMPERE POINT (seria Q) mają moduł Wi-Fi oparty na platformie Tuya. Pełny łańcuch komunikacji wygląda tak:

1. **Ładowarka ↔ router domowy:** Wi-Fi w paśmie **2,4 GHz** (standard 802.11 b/g/n, szyfrowanie WPA2). Moduły Tuya nie łączą się z sieciami 5 GHz — to częsta przyczyna problemów z parowaniem.
2. **Ładowarka ↔ chmura Tuya:** urządzenie utrzymuje stałe, szyfrowane połączenie z serwerami Tuya w przypisanym **centrum danych** (dla Polski: Central Europe, fizycznie w UE). Używany jest protokół **MQTT** (Message

Queuing Telemetry Transport — lekki protokół „publikuj/subskrybuj”, w którym urządzenie zgłasza zmiany do pośrednika, a pośrednik rozsyła je zainteresowanym) w tunelu **TLS** (Transport Layer Security — standardowe szyfrowanie transmisji, to samo „s” co w https). Treść komunikatów Tuya dodatkowo szyfruje własnym kluczem urządzenia. Tym kanałem płyną raporty punktów danych (DP) w górę i polecenia w dół.

3. **Aplikacja Smart Life ↔ chmura**: telefon rozmawia z chmurą przez **HTTPS** (zapytania **REST** — czyli zwykłe żądania sieciowe „pobierz/ustaw” na adresach API) oraz odbiera powiadomienia w czasie rzeczywistym. Konto aplikacji jest przypisane do centrum danych — dlatego wszędzie musi się zgadzać region.
4. **Home Assistant ↔ chmura — kanał 1 (konto aplikacji)**: oficjalna integracja Tuya oraz xtend_tuya w trybie podstawowym logują się **jak druga aplikacja** na Twoim koncie (stąd kod użytkownika + QR). Dostają listę urządzeń, specyfikacje w trybie instrukcji standardowych i strumień zmian stanów (połączenie zwrotne typu MQTT/WebSocket utrzymywane przez integrację). Zaleta: proste logowanie; wada: chmura pokazuje tym kanałem tylko „ugrzecznione”, okrojone opisy urządzeń.
5. **Home Assistant ↔ chmura — kanał 2 (OpenAPI, po etapie B)**: xtend_tuya z kluczami projektu (Client ID + Secret) odpytuje **OpenAPI** na adresie <https://openapi.tuya.eu.com>. Każde żądanie jest podpisywane funkcją skrótu **HMAC-SHA256** (podpis kryptograficzny z użyciem sekretu — serwer weryfikuje, że żądanie wysłał właściciel kluczy i że nikt go nie zmienił po drodze), a sesję utrzymuje krótkotrwały **token** („przepustka” ważna ok. 2 h, odświeżana automatycznie). Tym kanałem widać **pełne modele punktów danych** — zwłaszcza po przełączeniu produktów na tryb DP Instruction (etap C).

Ważny wniosek praktyczny: **cała integracja jest chmurowa**. Gdy padnie internet, HA traci kontakt z ładowarkami (encje przechodzą w stan „nieдоступny”), choć same ładowarki dalej ładują. Sterowanie lokalne (bez chmury) jest technicznie możliwe (protokół lokalny Tuya na porcie 6668, projekty typu LocalTuya), ale to osobny, bardziej wymagający projekt — wspomniany w rozdziale 8.

6.3. Platforma deweloperska Tuya — co czym jest

Projekt chmurowy (Cloud Project)

Twoja własna „aplikacja” po stronie API: ma klucze (Client ID + Secret), przypisane centrum danych i listę wykupionych/darmowych usług API. Projekt **AmperePoint HA** działa na darmowym planie **IoT Core Trial Edition** — wystarczającym dla domu/warsztatu, z limitem jednego centrum danych i okresową koniecznością przedłużania (patrz rozdział 9).

Usługi API (API Services)

moduły funkcjonalne projektu. Kluczowe: **IoT Core** (odczyt/sterowanie urządzeniami) i **Authorization Token Management** (wydawanie tokenów). Bez autoryzowanej usługi wywołania API kończą się błędem uprawnień (kod 1106 / „permission deny”).

Powiązanie konta aplikacji (Link App Account)

most między światem konsumenckim (Smart Life) a deweloperskim: po

zeskanowaniu QR projekt widzi urządzenia Twojego konta. Tryb **Automatic Link** łączy też urządzenia dodane w przeszłości.

PID (Product ID)

identyfikator *produktu* (modelu) — wspólny dla wszystkich egzemplarzy danego modelu. Definicja produktu (kategoria + zestaw DP) należy do producenta; stąd przypadek „Light Source” z p. 4.3.1.

Device ID

identyfikator konkretnego egzemplarza (np. `ladowarka_q11 = bf0b29260437b6d615ldyx`).

Tryb instrukcji (Standard/DP)

sposób, w jaki API prezentuje urządzenie projektowi — opisany w rozdziale 4.3.

6.4. Rola xtend_tuya i dodatku firmowego

xtend_tuya działa *obok* oficjalnej integracji Tuya i dokłada brakujące encje do tych samych urządzeń. Tryby pracy (nazewnictwo z dokumentacji): **OT+XT** — bez kluczy OpenAPI (nasz etap A; „większość” encji), **OT+XT+Cloud** — z kluczami (etapy B+C; „prawie wszystkie” encje). Autor używa nieoficjalnych rozwiązań („hacków”), dlatego dodatek nie wejdzie do oficjalnego HA i bywa wrażliwy na aktualizacje — to cena za pełne dane.

TuyaExtend **AmperePoint** (dodatek firmowy, katalog `custom_components/tuyaextend_amperepoint`) to **warstwa prezentacji**: w kreatorze wybiera się model (np. **AmperePoint Q22 OTA**), taryfę za kWh i mapuje istniejące encje (przełącznik, moc, energia sesji, energia całkowita, limit prądu, status, napięcia/prądy L1-L3). Dodatek liczy koszty, tłumaczy statusy na polski i buduje „ładne” urządzenie ładowarki. Konfigurować go warto **dopiero po** etapach B+C, gdy wszystkie encje źródłowe już istnieją.

7. Dokończenie konfiguracji — krok po kroku

Stan na koniec sesji: etap A zakończony, projekt deweloperski gotowy, część produktów przełączona na DP Instruction. Do zrobienia:

1. **Dokończyć etap C** (jeśli jeszcze nie zrobione): na `platform.tuya.com` → projekt **AmperePoint HA** → **Devices** → przycisk „...” → dla produktów **EV Charging Station, Q11_11KW_with_OTA, Q22 OTA** i **GPEV100**: ołówek → **DP Instruction** → **Save** → **OK**.
2. **Wpisać klucze w HA**: **Ustawienia** → **Urządzenia i usługi** → **Xtend Tuya** → **Konfiguruj** → formularz „Add Tuya OpenAPI credentials”: **odznaczyć** „Don’t ask...”, **Country** = **Poland**, **endpoint** = **Europe**, **Tuya IoT Access ID** i **Access Secret** skopiowane z karty Overview projektu (sekret odsłania ikona oka). Zatwierdzić.
3. **Zrestartować kontener** **homeassistant** w **Docker Desktop** i odczekać 1-2 minuty.
4. **Zweryfikować wynik**: **Ustawienia** → **Urządzenia i usługi** → **Xtend Tuya** → liczba encji przy urządzeniach powinna wzrosnąć (zwłaszcza Q22: docelowo

30+); w razie braków — odświeżyć stronę, ewentualnie „Załaduj ponownie” integrację (menu „...” przy wpisie Xtend Tuya).

8. Poradnik: następne kroki — weryfikacja, energia, automatyzacje

8.1. Porządki po konfiguracji

W **Rejestrze encji** (Ustawienia → Urządzenia i usługi → zakładka Rejestr encji) warto: nadać encjom czytelne nazwy i identyfikatory (np. `sensor.q22_moc`), wyłączyć encje zbędne (mniej szumu w bazie danych) oraz przypisać urządzenia do **obszarów** (np. „Pole”, „Serwis”). Duplikaty: oficjalna Tuya i `xtend_tuya` mogą pokazywać ten sam przełącznik dwa razy — to normalne; jeden z nich można wyłączyć w rejestrze.

8.2. Panel Energia

Ustawienia → Pulpity → **Energia**: w sekcji „Poszczególne urządzenia” dodać encje energii ładowarek (te sumujące kWh, zwykle „energia całkowita”/forward_energy_total). HA zacznie budować dzienne/miesięczne wykresy zużycia — podstawa do analiz i rozliczeń ładowania.

8.3. Dodatek firmowy TuyaExtend AmperePoint

Po pojawieniu się pełnych encji: Dodaj integrację → **TuyaExtend AmperePoint** → w kreatorze wybrać model (np. **AmperePoint Q22 OTA**), nazwę, taryfę (zł/kWh), walutę **PLN**, pozostawić domyślny próg mocy (0,25 kW), minuty bezczynności (3) i tryb energii sesji (Auto), a następnie zmapować encje źródłowe: przełącznik ładowania, moc, energię sesji, energię całkowitą, limit prądu, status, błąd oraz napięcia/prądy L1-L3. W razie pomyłki mapowanie zmienia się przyciskiem **Konfiguruj** bez usuwania integracji.

8.4. Automatyzacje — jak myśleć i trzy przykłady

Automatyzacja w HA składa się z **wyzwalacza** (trigger — „kiedy?”), opcjonalnych **warunków** (condition — „czy na pewno?”) i **akcji** (action — „co zrobić?”). Buduje się je wygodnie w kreatorze: Ustawienia → Automatyzacje i sceny → Utwórz automatyzację; poniżej zapis w formacie YAML (tekstowy format konfiguracji — wcięcia mają znaczenie). **Identyfikatory encji w przykładach są poglądowe** — należy podstawić własne z Rejestru encji.

1) Powiadomienie o zakończeniu ładowania (wymaga aplikacji Home Assistant Companion na telefonie, która dodaje usługę notify):

```
alias: Powiadom o koncu ładowania Q22
trigger:
  - platform: state
    entity_id: sensor.q22_status
    from: "charging"
    to: "charger_free"
```

```

action:
- service: notify.mobile_app_telefon_dawida
  data:
    title: "Ładowarka Q22"
    message: "Ładowanie zakończone."

```

2) Ładowanie tylko w taniej taryfie (start 22:00, stop 6:00):

```

alias: Q22 tania taryfa
trigger:
- platform: time
  at: "22:00:00"
  id: start
- platform: time
  at: "06:00:00"
  id: stop
action:
- choose:
  - conditions: [{ condition: trigger, id: start }]
    sequence:
  - service: switch.turn_on
    target: { entity_id: switch.q22_przelacznik }
  - conditions: [{ condition: trigger, id: stop }]
    sequence:
  - service: switch.turn_off
    target: { entity_id: switch.q22_przelacznik }

```

3) Alarm przy błędzie ładowarki:

```

alias: Alarm błędu ładowarki
trigger:
- platform: state
  entity_id: sensor.q22_blad
condition:
- condition: template
  value_template: "{{ trigger.to_state.state not in ['0', 'ok', 'unknown', 'unavailable'] }}"
action:
- service: notify.mobile_app_telefon_dawida
  data:
    title: "UWAGA: błąd ładowarki"
    message: "Kod błędu: {{ trigger.to_state.state }}"

```

Pomysły na dalsze automatyzacje: obniżanie limitu prądu (encja typu *number*, usługa `number.set_value`) w godzinach szczytu; raport dzienny energii; ładowanie nadwyżką z fotowoltaiki (po zintegrowaniu falownika z HA — wtedy wyzwaczem jest moc eksportu do sieci).

8.5. Pulpit (dashboard)

Na pulpicie warto zestawić kartę **wskaźnika** (gauge) z mocą chwilową, kartę **historii** z energią sesji, kartę **encji** ze statusem/limitem prądu/przełącznikiem. Karty dodaje się z poziomu pulpitu przyciskiem „Edytuj”.

8.6. Kopie zapasowe i aktualizacje

W instalacji kontenerowej najpewniejszą kopią zapasową jest **kopia całego katalogu** `C:\HomeAssistant\config` (najlepiej przy zatrzymanym kontenerze).

Aktualizacja HA = pobranie nowego obrazu kontenera (docker pull) i ponowne utworzenie kontenera; aktualizacje xtend_tuya — przez HACS, ale rozważnie: po każdej większej aktualizacji HA sprawdzić w HACS, czy xtend_tuya ma wersję zgodną (dodatek używa nieoficjalnych mechanizmów i bywa czuły na zmiany).

8.7. Kierunek na przyszłość: sterowanie lokalne

Jeżeli zależność od chmury zacznie przeszkadzać (opóźnienia, przerwy internetu), istnieje ścieżka **LocalTuya** / **tuya-local**: sterowanie po sieci lokalnej z użyciem *kluczy lokalnych* urządzeń (local key — indywidualny klucz szyfrowania, do odczytania przez API projektu deweloperskiego). Wymaga stałych adresów IP dla ładowarek w routerze i ręcznego mapowania DP; bywa pracochłonna, ale uniezależnia podstawowe funkcje od internetu. Na dziś — opcja, nie konieczność.

9. Przewidywane problemy i proponowane rozwiązania

Objaw	Prawdopodobna przyczyna	Rozwiązanie
Po wpisaniu kluczy brak nowych encji	Klucze wpisane, ale integracja nie odświeżyła danych	Restart kontenera; potem menu „...” przy wpisie Xtend Tuya → Załaduj ponownie. Sprawdzić logi: Ustawienia → System → Logi, filtr „xtend”.
Błąd „sign invalid” w logach	Zły Client ID/Secret (literówka, spacja) albo zły endpoint	Skopiować klucze ponownie z karty Overview; endpoint musi być Europe (openapi.tuya.eu.com).
Błąd „permission deny” / kod 1106	Nieautoryzowana usługa API w projekcie albo wygasła subskrypcja	Na platformie: Service API → sprawdzić, czy IoT Core i Authorization są aktywne; w razie potrzeby Authorize/Renew.
Encje przestają się aktualizować po kilku miesiącach	IoT Core Trial Edition wygasa (typowo co pół roku)	Na platformie: Cloud → projekt → IoT Core → <i>Extend Trial</i> (bezpłatne przedłużenie). Wpisać do kalendarza przypomnienie.
Urządzenia w projekcie „No data found” albo puste listy	Przeglądarka zgubiła region (przekierowania platform ↔ eu.platform)	Odświeżyć stronę; upewnić się, że w prawym górnym rogu wybrane jest Central Europe Data Center.

Objaw	Prawdopodobna przyczyna	Rozwiązanie
QR z platformy nie daje się zeskanować / błąd autoryzacji	Konto aplikacji jest w innym centrum danych niż projekt	Sprawdzić region konta (dokument „App account data center distribution” podlinkowany przy QR); projekt musi mieć ten sam DC.
Wszystkie encje „niedostępne”	Przerwa internetu lub awaria chmury Tuya; ładowarka offline	Sprawdzić internet i stan urządzenia w Smart Life. Integracja jest chmurowa — bez internetu nie działa (docelowo: LocalTuya).
Po aktualizacji HA integracja Xtend Tuya nie startuje	Niezgodność wersji dodatku z nowym HA	W HACS zaktualizować xtend_tuya; jeśli poprawki brak — tymczasowo wrócić do poprzedniego obrazu HA. Przed aktualizacją HA sprawdzać zgodność.
Zdublowane encje (dwa przełączniki tej samej ładowarki)	Oficjalna Tuya i Xtend Tuya wystawiają ten sam element	Stan normalny; zbędną encję wyłączyć w Rejestrze encji (nie usuwać integracji).
„AmperePoint EV Charger” wciąż ma 1 encję	Produkt zarejestrowany przez producenta jako Light Source z jednym DP (switch_led)	Zgłosić producentowi (PID gbmngp0loofmhbjc) prośbę o poprawną kategorię i pełny zestaw DP. Po stronie HA nic się nie da zrobić.
Ładowarka nie łączy się z Wi-Fi po zmianie routera/hasła	Moduły Tuya wymagają pasma 2,4 GHz i ponownego parowania	Włączyć sieć 2,4 GHz, sparować urządzenie ponownie w Smart Life; urządzenia wrócą do HA automatycznie (Automatic Link).
Obawa: „czy tryb DP zepsuł mi aplikację?”	Ostrzeżenie platformy przy zmianie trybu instrukcji	Nie — zmiana dotyczy tylko sposobu prezentacji urządzeń w <i>tym projekcie API</i> ; aplikacja Smart Life i ładowarki działają bez zmian.
Wyciek kluczy (Client Secret)	Klucz zapisany w miejscu współdzielonym	Na platformie można zresetować sekret (Overview → ikona przy kluczu); po resecie zaktualizować go w HA.

10. Słowniczek pojęć i skrótów

API (Application Programming Interface)

interfejs programistyczny — „okienko podawcze”, przez które programy proszą usługę o dane lub działania.

Centrum danych (Data Center, DC)

serwerownia regionu, do której przypisane jest konto Tuya; u nas Central Europe.

Client ID / Access ID i Client Secret / Access Secret

para „login + hasło” projektu deweloperskiego do podpisywania żądań API.

Config entry (wpis konfiguracyjny)

zapisany wynik kreatora integracji w HA; bez niego integracja nie działa.

Config flow

kreator konfiguracji integracji (okienka „dalej/dalej” w HA).

Docker / kontener

sposób uruchamiania programów w odizolowanych „pudełkach”; HA działa jako kontener homeassistant.

DP (data point, punkt danych)

pojedyncza zmienna urządzenia Tuya (np. moc, status); ma numer, kod i typ.

Encja (entity)

pojedynczy czujnik/przełącznik/wartość w HA.

Endpoint

adres serwera API (u nas <https://openapi.tuya.eu.com>).

HA Home Assistant.

HACS

społecznościowy instalator dodatków do HA.

HMAC-SHA256

podpis kryptograficzny żądań API wykonywany sekretem projektu.

HTTPS/TLS

szyfrowana transmisja w sieci (kłódka w przeglądarce).

Integracja

moduł HA łączący go z usługą/urządzeniami.

MQTT

lekki protokół „publikuj/subskrybuj” używany przez urządzenia IoT do stałej łączności z chmurą.

OpenAPI (Tuya)

oficjalne API deweloperskie chmury Tuya.

OTA (Over-The-Air)

aktualizacja oprogramowania urządzenia „przez powietrze” (zdalnie); człon nazw produktów Q OTA.

PID (Product ID)

identyfikator modelu produktu w Tuya; Device ID — identyfikator egzemplarza.

QR (kod QR)

kwadratowy kod kreskowy skanowany telefonem; tu: do autoryzacji logowań.

REST

styl API: proste żądania „pobierz/ustaw” pod adresami HTTP.

SQLite

plikowa baza danych, w której HA zapisuje historię stanów.

Token

tymczasowa „przepustka” do API wydawana po uwierzytelnieniu; odświeżana automatycznie.

User Code (kod użytkownika)

identyfikator konta aplikacji Smart Life używany przy logowaniu integracji przez QR.

WSL2

lekka maszyna wirtualna z Linuksem, w której Docker Desktop uruchamia kontenery na Windows.

YAML

tekstowy format konfiguracji automatyzacji HA (wcięcia mają znaczenie).

Koniec dokumentu — wersja 1, 2026-07-05. Kolejne wersje oznaczać v2, v3... bez usuwania poprzednich.